PYTHON PROGRAMMING CASE STUDIES

Problem • Algorithm • Flowchart • Python Code • Sample Output • Explanation

Programs 11 to 20

Student Information

Student Name:	Nakka Bhargavi
Roll Number:	26000183
Section:	CSE-3

Case Studies Included

11. Examination Result and Grace-Mark Validation
12. Attendance Recovery Computation
13. ATM Transaction Sequence Simulation
14. Warehouse Inventory Reconciliation
15. Smart Parking Fee Management
16. Multi-Day Water Tank Simulation
17. Student Performance Trend Analysis
18. Fuel Consumption and Route Comparison
19. Multi-Product Shopping Budget Constraint
20. University Admission Decision Engine

Each case study contains the problem statement, algorithm, a coloured flowchart diagram, Python code, sample output, and explanation.

Program 11: Examination Result and Grace-Mark Validation

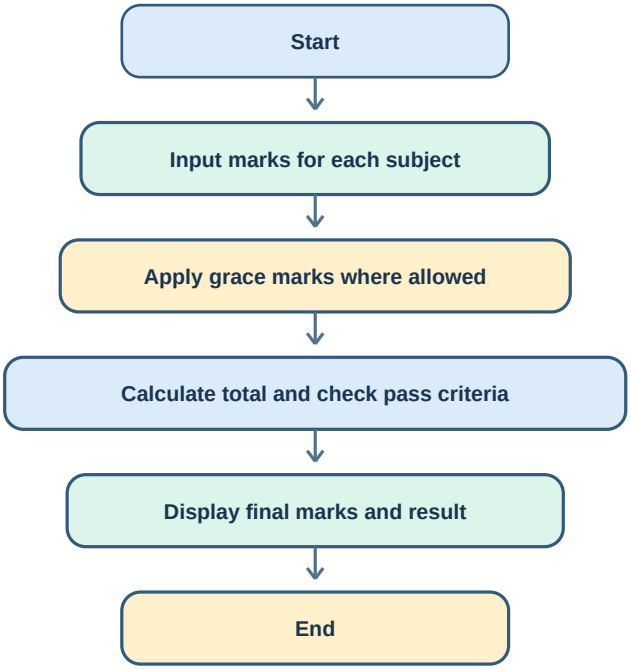
1. Problem Statement

Develop a program to process marks in multiple subjects, apply a stated grace-mark policy, calculate total marks, and determine the final result.

2. Algorithm

1. Start the program.
2. Input the number of subjects and marks.
3. Add 5 grace marks to any subject mark from 30 to 34.
4. Calculate the total of the adjusted marks.
5. If every subject mark is at least 35, display Pass; otherwise, display Fail.
6. Stop.

3. Flowchart



4. Python Code

```
marks = []
n = int(input("Enter number of subjects: "))

for i in range(n):
    m = int(input("Enter marks out of 100: "))
    marks.append(m)

for i in range(len(marks)):
    if 30 <= marks[i] < 35:
        marks[i] = min(100, marks[i] + 5)

total = sum(marks)
print("Final Marks:", marks)
print("Total Marks:", total)

if all(m >= 35 for m in marks):
    print("Result: Pass")
else:
    print("Result: Fail")
```

5. Sample Output

```
Enter number of subjects: 3
Enter marks out of 100: 34
Enter marks out of 100: 76
```

```
Enter marks out of 100: 80
Final Marks: [39, 76, 80]
Total Marks: 195
Result: Pass
```

6. Explanation / Conclusion

Grace-mark rule used for this example: add 5 marks when a subject mark is between 30 and 34. Confirm the exact policy with your faculty.

Program 12: Attendance Recovery Computation

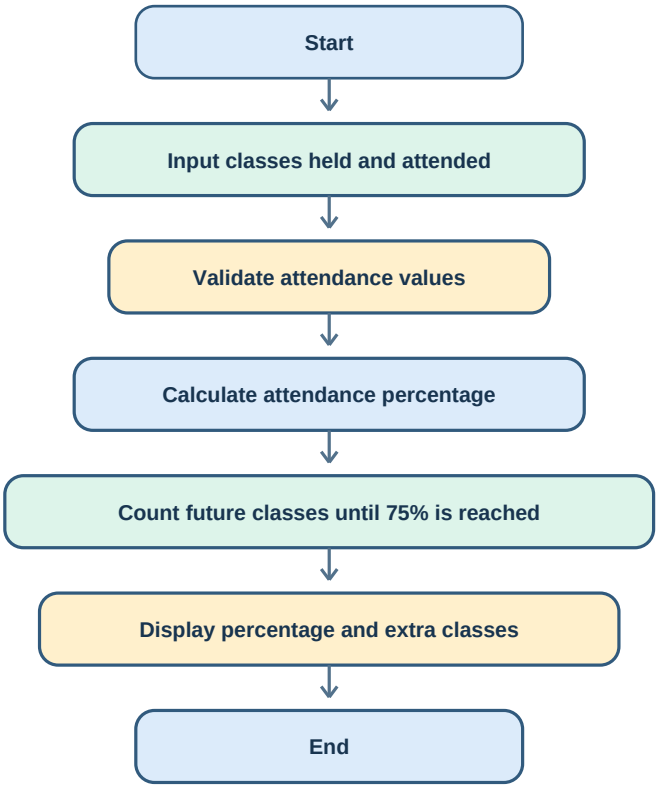
1. Problem Statement

Calculate the current attendance percentage and find the minimum number of additional classes a student must attend to reach 75%.

2. Algorithm

1. Start the program.
2. Input total classes held and classes attended.
3. Check that the values are valid.
4. Calculate attendance = attended / held × 100.
5. While attendance is below 75%, increase both attended and held by one.
6. Display the number of extra classes and final percentage.
7. Stop.

3. Flowchart



4. Python Code

```
held = int(input("Enter total classes held: "))
attended = int(input("Enter classes attended: "))

if held <= 0 or attended < 0 or attended > held:
    print("Invalid attendance data")
else:
    percentage = attended / held * 100
    print("Attendance Percentage:", round(percentage, 2), "%")
    classes = 0

    while percentage < 75:
        attended += 1
        held += 1
        classes += 1
        percentage = attended / held * 100

    print("Additional classes required:", classes)
    print("Final Attendance:", round(percentage, 2), "%")
```

5. Sample Output

```
Enter total classes held: 40
Enter classes attended: 25
Attendance Percentage: 62.5 %
Additional classes required: 20
Final Attendance: 75.0 %
```

6. Explanation / Conclusion

The example assumes that every additional class is attended.

Program 13: ATM Transaction Sequence Simulation

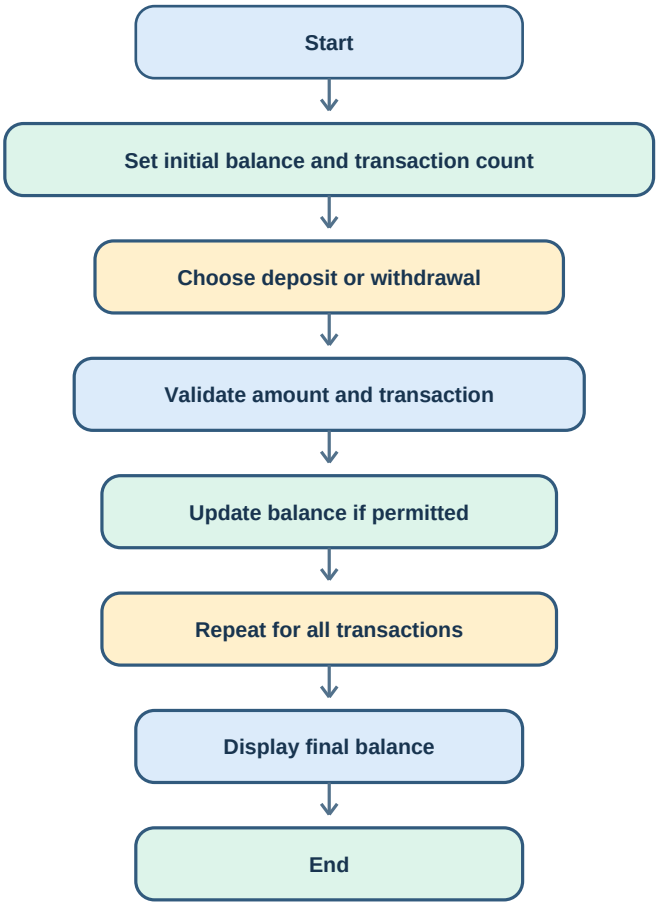
1. Problem Statement

Simulate a sequence of deposits and withdrawals, maintain the balance, reject invalid transactions, and display the final balance.

2. Algorithm

1. Start with a balance of Rs. 1000.
2. Input the number of transactions.
3. For each transaction, choose deposit or withdrawal and enter the amount.
4. Reject amounts that are zero or negative.
5. Add deposits; subtract withdrawals only when the balance and limit permit.
6. Display the final balance.
7. Stop.

3. Flowchart



4. Python Code

```
balance = 1000
n = int(input("Enter number of transactions: "))

for i in range(n):
    print("1. Deposit")
    print("2. Withdraw")
    choice = int(input("Enter your choice: "))
    amount = float(input("Enter amount: "))

    if amount <= 0:
        print("Invalid amount")
    elif choice == 1:
        balance += amount
```

```
        print("Deposit successful")
    elif choice == 2:
        if amount <= balance and amount <= 5000:
            balance -= amount
            print("Withdrawal successful")
        else:
            print("Insufficient balance or limit exceeded")
    else:
        print("Invalid choice")

print("Final Balance:", balance)
```

5. Sample Output

```
Enter number of transactions: 2
1. Deposit
2. Withdraw
Enter your choice: 1
Enter amount: 500
Deposit successful
1. Deposit
2. Withdraw
Enter your choice: 2
Enter amount: 200
Withdrawal successful
Final Balance: 1300.0
```

6. Explanation / Conclusion

The sample uses an initial balance of Rs. 1000 and a maximum single-withdrawal amount of Rs. 5000.

Program 14: Warehouse Inventory Reconciliation

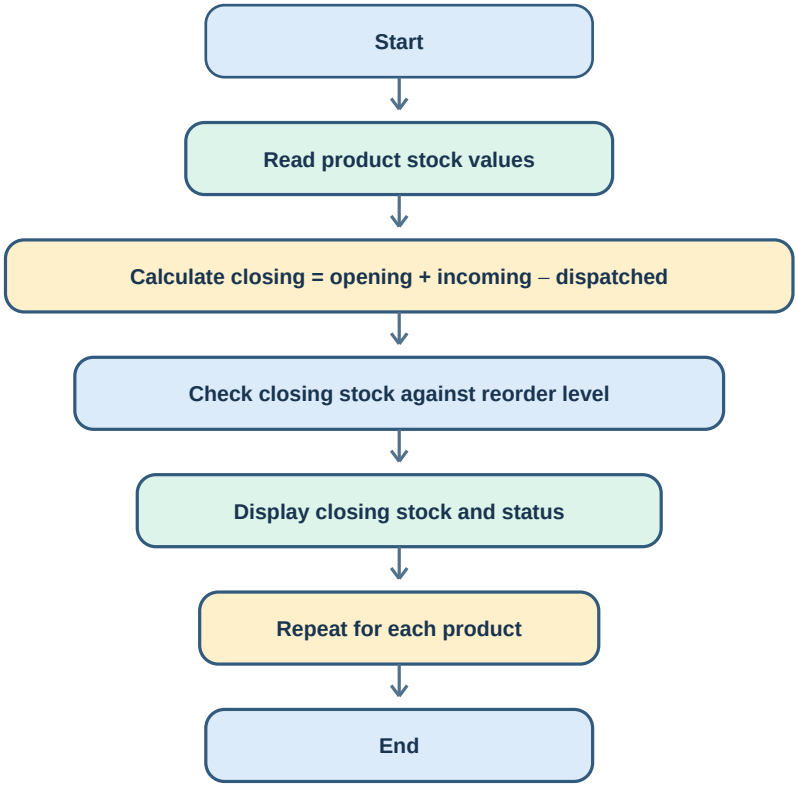
1. Problem Statement

Calculate closing stock for several products using opening stock, incoming stock, and dispatched stock, then identify items requiring reordering.

2. Algorithm

1. Start the program.
2. Store product names, opening stock, incoming quantities, and dispatched quantities in lists.
3. Calculate closing stock for each product.
4. If closing stock is below 20, display Reorder required.
5. Otherwise, display Stock sufficient.
6. Stop.

3. Flowchart



4. Python Code

```
products = ["Pens", "Books", "Pencils"]
opening = [100, 50, 80]
incoming = [20, 10, 30]
dispatched = [40, 25, 50]

for i in range(len(products)):
    closing = opening[i] + incoming[i] - dispatched[i]
    print("Product:", products[i])
    print("Closing Stock:", closing)

    if closing < 20:
        print("Status: Reorder required")
    else:
        print("Status: Stock sufficient")
```

5. Sample Output

```
Product: Pens
Closing Stock: 80
Status: Stock sufficient
Product: Books
```

```
Closing Stock: 35
Status: Stock sufficient
Product: Pencils
Closing Stock: 60
Status: Stock sufficient
```

6. Explanation / Conclusion

The product quantities and reorder level are sample values. The program can be adapted to use input values.

Program 15: Smart Parking Fee Management

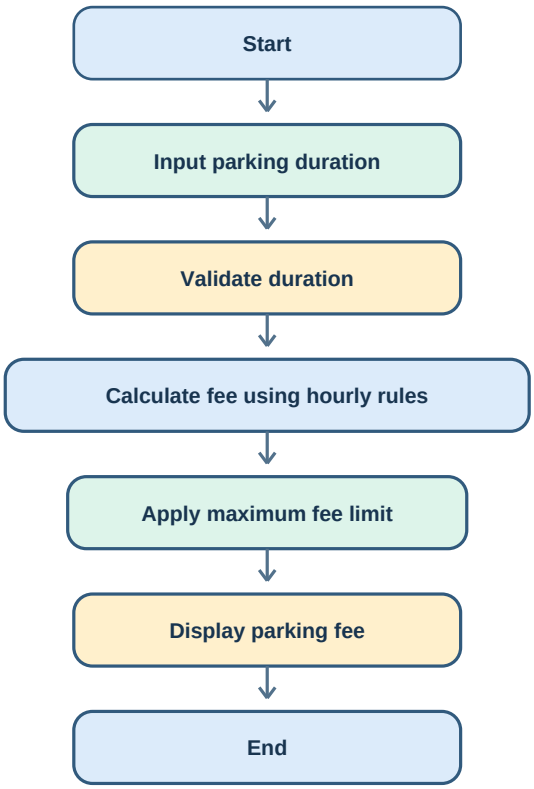
1. Problem Statement

Calculate parking fees from the number of hours parked, apply different hourly rates, and enforce a maximum daily fee.

2. Algorithm

1. Start the program.
2. Input the number of parking hours.
3. If the duration is not positive, display an error.
4. Charge Rs. 20 per hour for the first two hours and Rs. 10 per additional hour.
5. Limit the fee to Rs. 150.
6. Display the fee and stop.

3. Flowchart



4. Python Code

```
hours = int(input("Enter parking hours: "))

if hours <= 0:
    print("Invalid parking duration")
else:
    if hours <= 2:
        fee = hours * 20
    else:
        fee = 40 + (hours - 2) * 10

    if fee > 150:
        fee = 150

    print("Parking Fee: Rs.", fee)
```

5. Sample Output

```
Enter parking hours: 5
Parking Fee: Rs. 70
```

6. Explanation / Conclusion

Illustrative tariff: Rs. 20 per hour for the first two hours, then Rs. 10 per extra hour, capped at Rs. 150.

Program 16: Multi-Day Water Tank Simulation

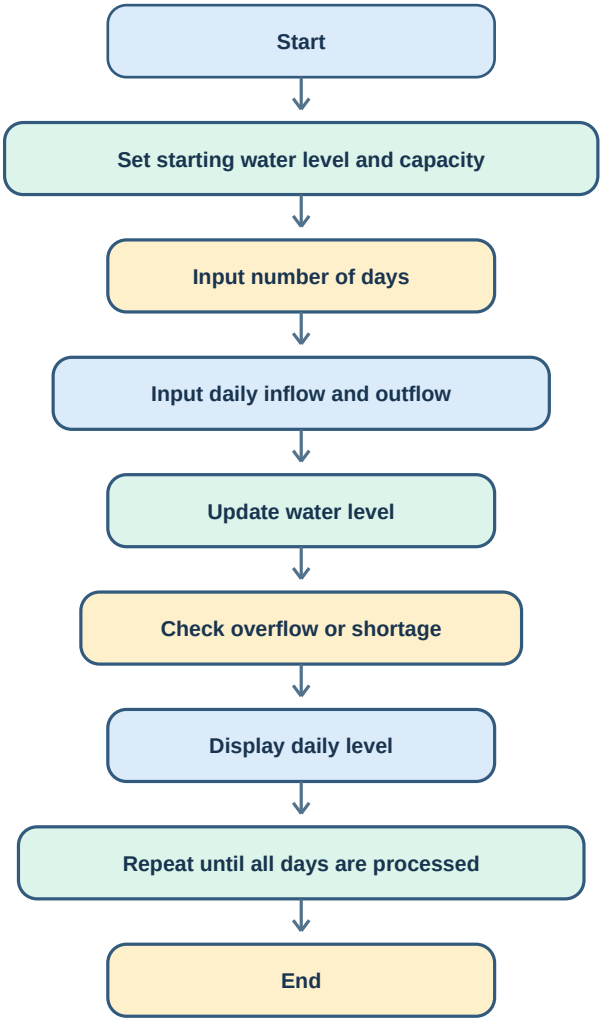
1. Problem Statement

Update a tank's water level each day based on inflow and outflow, and report overflow or water shortage.

2. Algorithm

1. Start with a water level of 500 units and a capacity of 1000 units.
2. Input the number of days.
3. For each day, input inflow and outflow.
4. Update the level by adding inflow and subtracting outflow.
5. If the level exceeds capacity, set it to capacity and report overflow.
6. If the level drops below zero, set it to zero and report shortage.
7. Display each day's level and stop after all days.

3. Flowchart



4. Python Code

```
level = 500
capacity = 1000
days = int(input("Enter number of days: "))

if days < 0:
    print("Invalid number of days")
else:
    for i in range(days):
        inflow = int(input("Enter water inflow: "))
        outflow = int(input("Enter water outflow: "))
```

```
    if inflow < 0 or outflow < 0:
        print("Invalid inflow/outflow")
        continue

    level = level + inflow - outflow

    if level > capacity:
        level = capacity
        print("Overflow detected")
    if level < 0:
        level = 0
        print("Water shortage detected")

    print("Day", i + 1, "Water Level:", level)
```

5. Sample Output

```
Enter number of days: 2
Enter water inflow: 200
Enter water outflow: 100
Day 1 Water Level: 600
Enter water inflow: 500
Enter water outflow: 200
Day 2 Water Level: 900
```

6. Explanation / Conclusion

The initial level and tank capacity are illustrative values, measured in generic water units.

Program 17: Student Performance Trend Analysis

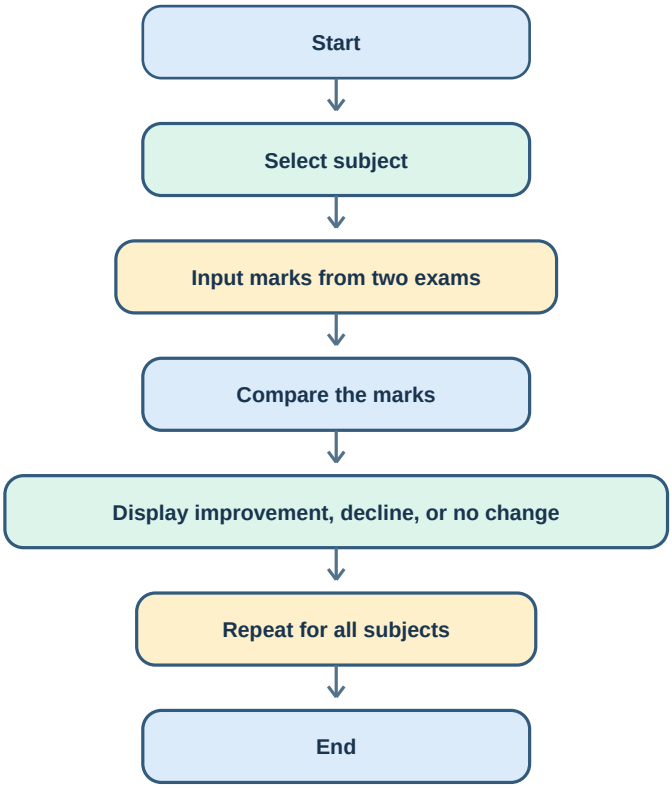
1. Problem Statement

Compare marks from two examinations for multiple subjects and identify whether performance improved, declined, or remained unchanged.

2. Algorithm

1. Start the program.
2. Process each subject in the subject list.
3. Input marks from the first and second examinations.
4. If second marks are higher, display improved; if lower, display declined; otherwise, display unchanged.
5. Repeat for every subject and stop.

3. Flowchart



4. Python Code

```
subjects = ["Python", "Maths", "English"]

for subject in subjects:
    first = int(input("Enter first exam marks for " + subject + ": "))
    second = int(input("Enter second exam marks for " + subject + ": "))

    print(subject)
    if second > first:
        print("Performance improved")
    elif second < first:
        print("Performance declined")
    else:
        print("Performance unchanged")
```

5. Sample Output

```
Enter first exam marks for Python: 65
Enter second exam marks for Python: 78
Python
Performance improved
Enter first exam marks for Maths: 80
Enter second exam marks for Maths: 72
```

```
Maths
Performance declined
Enter first exam marks for English: 70
Enter second exam marks for English: 70
English
Performance unchanged
```

6. Explanation / Conclusion

Enter marks between 0 and 100 for each exam. This example compares each subject separately.

Program 18: Fuel Consumption and Route Comparison

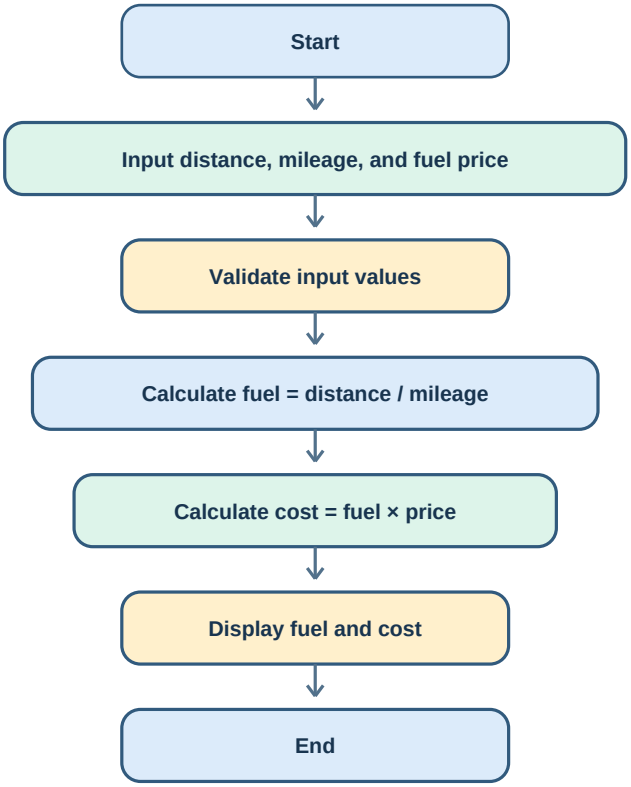
1. Problem Statement

Calculate fuel required and estimated journey cost using distance, vehicle mileage, and fuel price.

2. Algorithm

1. Start the program.
2. Input journey distance, mileage, and fuel price.
3. Check that distance and mileage are positive and price is not negative.
4. Calculate fuel required by dividing distance by mileage.
5. Multiply fuel required by fuel price to calculate cost.
6. Display the results and stop.

3. Flowchart



4. Python Code

```
distance = float(input("Enter journey distance in km: "))
mileage = float(input("Enter vehicle mileage in km per litre: "))
price = float(input("Enter fuel price per litre: "))

if distance <= 0 or mileage <= 0 or price < 0:
    print("Invalid input")
else:
    fuel = distance / mileage
    cost = fuel * price
    print("Fuel Required:", round(fuel, 2), "litres")
    print("Journey Cost: Rs.", round(cost, 2))
```

5. Sample Output

```
Enter journey distance in km: 150
Enter vehicle mileage in km per litre: 15
Enter fuel price per litre: 100
Fuel Required: 10.0 litres
Journey Cost: Rs. 1000.0
```

6. Explanation / Conclusion

This example calculates one vehicle's cost. Comparing routes or multiple vehicles would require additional inputs.

Program 19: Multi-Product Shopping Budget Constraint

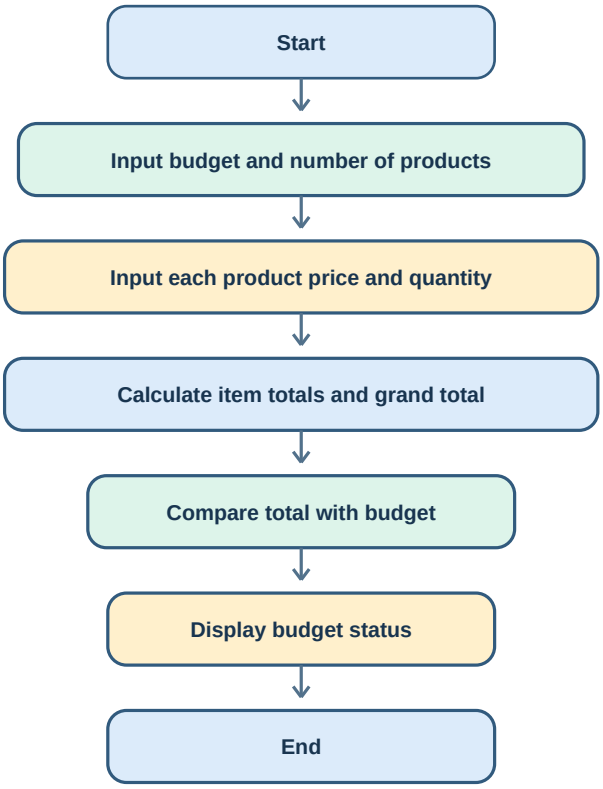
1. Problem Statement

Calculate the total cost of a shopping basket containing multiple products and check whether the total stays within the given budget.

2. Algorithm

1. Start the program.
2. Input the shopping budget and number of products.
3. For each product, input price and quantity.
4. Calculate each product cost as price × quantity and add it to the total.
5. Compare the total with the budget.
6. Display whether the shopping is within budget or the amount over budget.
7. Stop.

3. Flowchart



4. Python Code

```
budget = float(input("Enter your budget: "))
n = int(input("Enter number of products: "))

if budget < 0 or n < 0:
    print("Invalid input")
else:
    total = 0
    for i in range(n):
        price = float(input("Enter product price: "))
        quantity = int(input("Enter quantity: "))

        if price < 0 or quantity < 0:
            print("Invalid price or quantity")
        else:
            total += price * quantity

    print("Total Bill: Rs.", round(total, 2))
    if total <= budget:
```

```
        print("Shopping is within budget")
    else:
        print("Shopping exceeds budget")
        print("Amount to reduce: Rs.", round(total - budget, 2))
```

5. Sample Output

```
Enter your budget: 1000
Enter number of products: 2
Enter product price: 250
Enter quantity: 2
Enter product price: 100
Enter quantity: 3
Total Bill: Rs. 800.0
Shopping is within budget
```

6. Explanation / Conclusion

The program reports the amount over budget; it does not automatically select which products to remove.

Program 20: University Admission Decision Engine

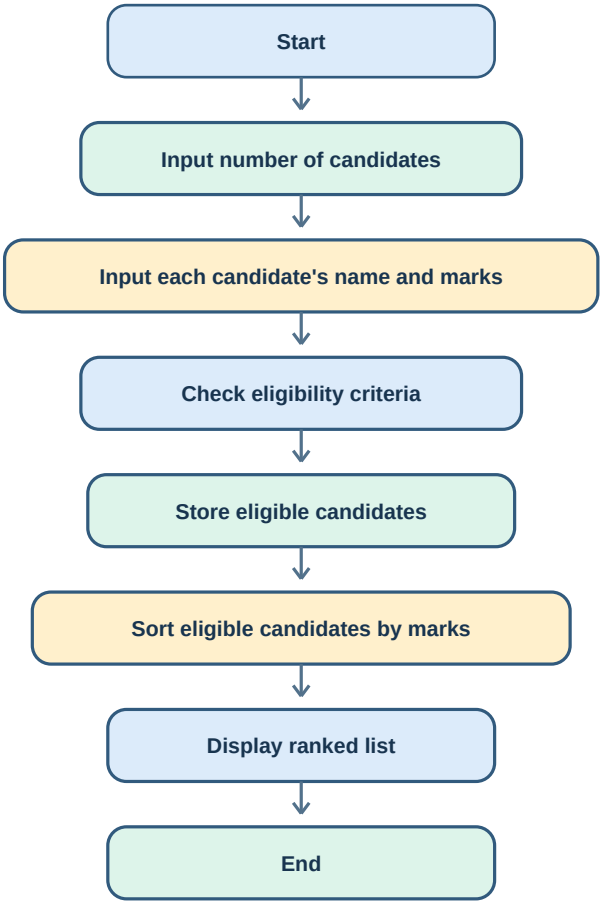
1. Problem Statement

Check candidates against aggregate and subject-mark thresholds, then rank eligible candidates by aggregate marks.

2. Algorithm

1. Start the program.
2. Input the number of candidates.
3. For each candidate, input name, aggregate marks, and required-subject marks.
4. A candidate is eligible when aggregate marks are at least 70 and subject marks are at least 60.
5. Store eligible candidates and sort them from highest to lowest aggregate marks.
6. Display the ranked list or a message if no candidates qualify.
7. Stop.

3. Flowchart



4. Python Code

```
candidates = []
n = int(input("Enter number of candidates: "))

for i in range(n):
    name = input("Enter candidate name: ")
    marks = int(input("Enter aggregate marks out of 100: "))
    subject = int(input("Enter required subject marks: "))

    if marks >= 70 and subject >= 60:
        candidates.append((name, marks))

candidates.sort(key=lambda x: x[1], reverse=True)

print("\nEligible Candidates Ranked by Marks:")
if len(candidates) == 0:
```

```
    print("No eligible candidates")
else:
    for rank, candidate in enumerate(candidates, start=1):
        print(rank, candidate[0], "-", candidate[1])
```

5. Sample Output

```
Enter number of candidates: 3
Enter candidate name: Anu
Enter aggregate marks out of 100: 85
Enter required subject marks: 70
Enter candidate name: Ravi
Enter aggregate marks out of 100: 68
Enter required subject marks: 75
Enter candidate name: Sita
Enter aggregate marks out of 100: 92
Enter required subject marks: 80

Eligible Candidates Ranked by Marks:
1 Sita - 92
2 Anu - 85
```

6. Explanation / Conclusion

The eligibility thresholds are sample assumptions. Use the actual aggregate, subject, and tie-breaking rules provided by your university.